Laboratorio 4

Ingeniería del software III
Grupo G1

Estudiantes:
2211881 Edgar Santiago Ariza Garcia
2211860 Javier Alejandro Silva Murillo
2211590 Oscar Fabian Fuentes Zuñiga

Presentado a:
Darío Alejandro Riaño Velandia

Bucaramanga
2026

1. Introducción

En este laboratorio se implementó una arquitectura basada en microservicios para el sistema Demo Académico, separando las funcionalidades de autenticación y consulta de información en dos servicios independientes.

El microservicio “auth-service” se encarga de gestionar la autenticación de usuarios, la generación y validación de tokens JWT. Por otra parte, “consulta_service” proporciona los endpoints para consultar la información académica y verifica la autorización de las solicitudes mediante la comunicación con el servicio de autenticación.

La ejecución de los servicios se realizó tanto de manera independiente como mediante Docker Compose, permitiendo comprobar su funcionamiento y la comunicación entre ambos componentes.

2. Objetivos

Objetivo general

Implementar y verificar una arquitectura de microservicios que permita separar los procesos de autenticación y consulta de información, utilizando tokens JWT para controlar el acceso a los recursos.

Objetivos específicos

- Configurar el microservicio de autenticación para gestionar usuarios y generar tokens JWT.
- Implementar el microservicio de consulta para acceder a los recursos académicos.
- Establecer la comunicación entre ambos microservicios.
- Implementar mecanismos de validación de tokens para proteger los endpoints
- Verificar el funcionamiento de los servicios mediante Swagger y Docker Compose.

3. Implementación

3.1 Microservicio de autenticación

Se desarrolló el microservicio “auth-service”, encargado de gestionar el inicio de sesión y la validación de tokens.

El servicio utiliza una base de datos H2 en memoria para almacenar los usuarios de prueba. Las contraseñas se gestionan mediante BCrypt y cuando las credenciales son correctas, se genera un token JWT que permite identificar el usuario y su rol.
Se configuraron los siguientes usuarios para realizar las pruebas

Usuario	**Contraseña**	**Rol**
admin	Admin2026*	ADMIN
docente	Docente2026*	DOCENTE
estudiante1	Estu2026*	ESTUDIANTE

3.2 Microservicio de consulta

Se configuró el microservicio “consulta-service”, encargado de exponer los endpoints relacionados con las consultas académicas.

Este servicio utiliza OpenFeign para comunicarse con “auth-service” y verificar la validez de los tokens recibidos. De esta manera, las solicitudes a los endpoints protegidos requieren un token válido para permitir el acceso a la información.

3.3 Contenerización y comunicación

Se configuraron archivos Dockerfile para cada microservicio y un archivo “docker-compose.yml” para construir y ejecutar ambos servicios dentro de contenedores independientes.

La configuración establece los puertos 8081 para “auth-service” y 8082 para “consulta-service”. Además, se configuró la variable de entorno SERVICES_AUTH_URL con la dirección http://auth-service:8081 , permitiendo que el microservicio de consulta se comunique con el servicio de autenticación a través de la red interna de Docker.

4. Pruebas y resultados

Se realizó una solicitud al endpoint “POST /auth/login“ utilizando credenciales válidas. El servicio respondió correctamente y generó un token JWT asociado al usuario autenticado.

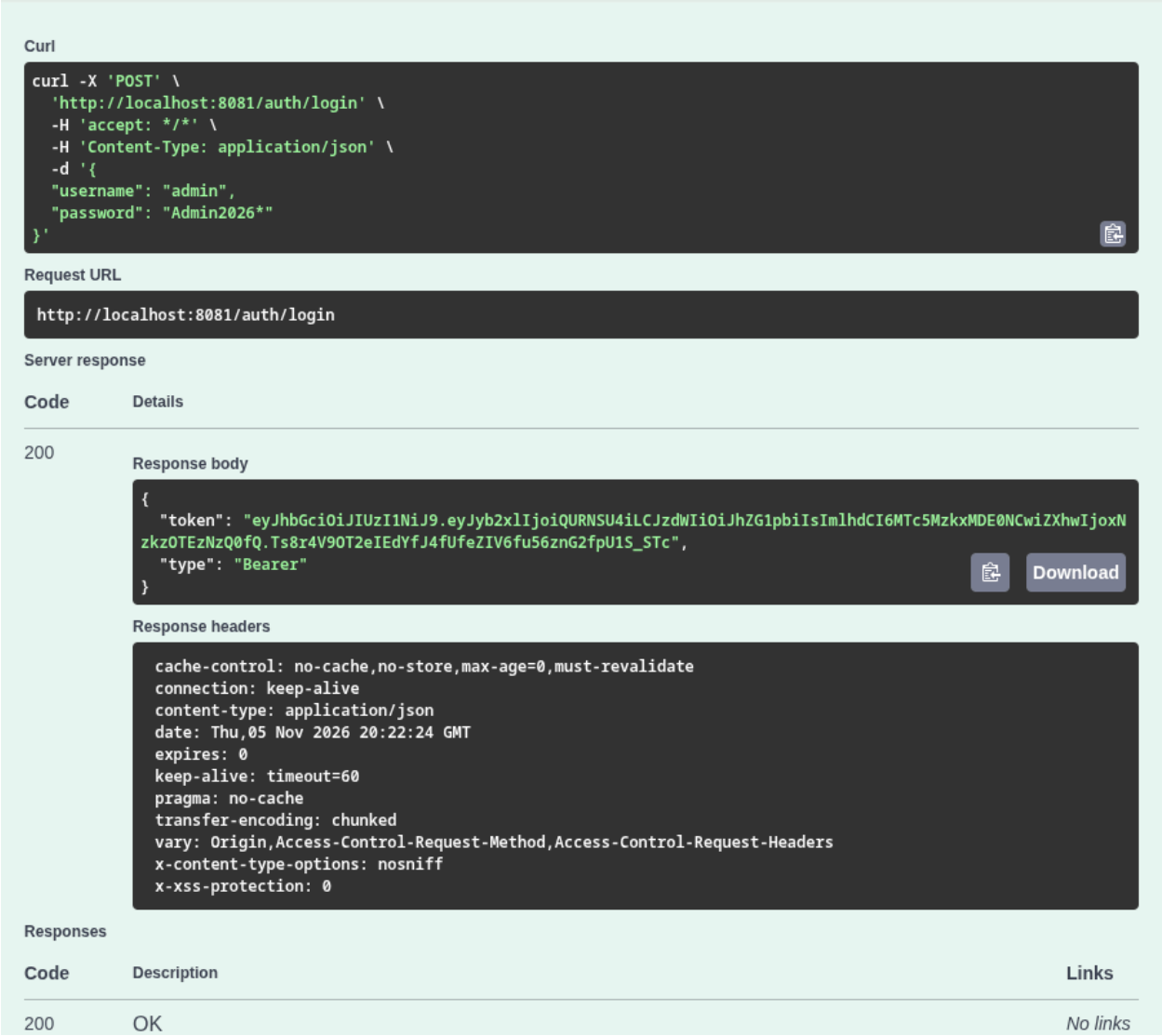


Figura 1 Inicio de sesión exitoso

4.2 Inicio de sesión fallido

Se realizó una solicitud al endpoint “POST /auth/login“ utilizando credenciales incorrectas. El servicio rechazó la autenticación y no generó ningún token de acceso.

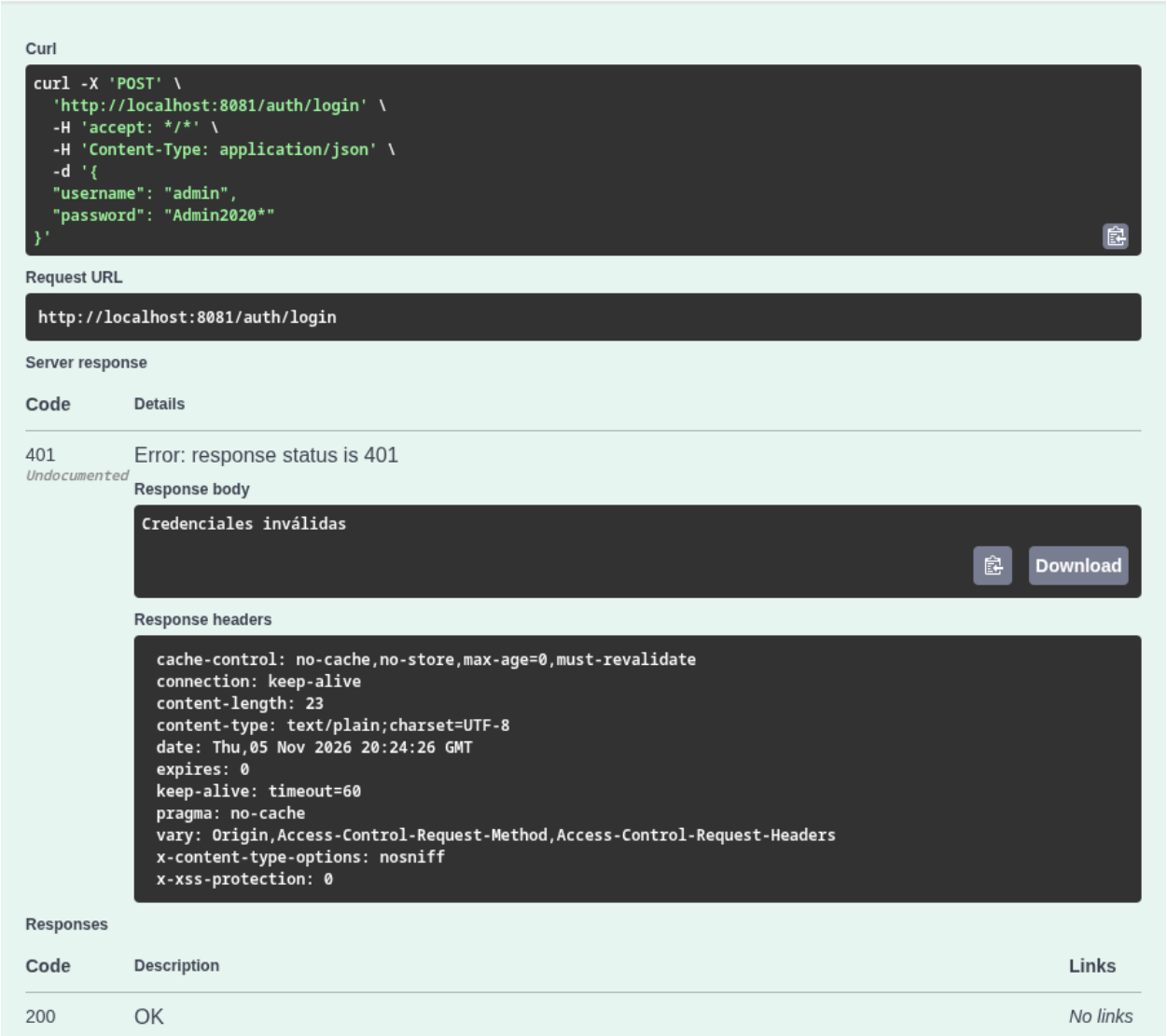


Figura 2 Inicio de sesión con credenciales incorrectas

4.3 Validación del token

Se utilizó el endpoint “GET /auth/validate” para comprobar la validez de un token JWT generado previamente. El servicio verificó el token y retornó la información correspondiente al usuario autenticado, incluyendo su nombre de usuario y rol.

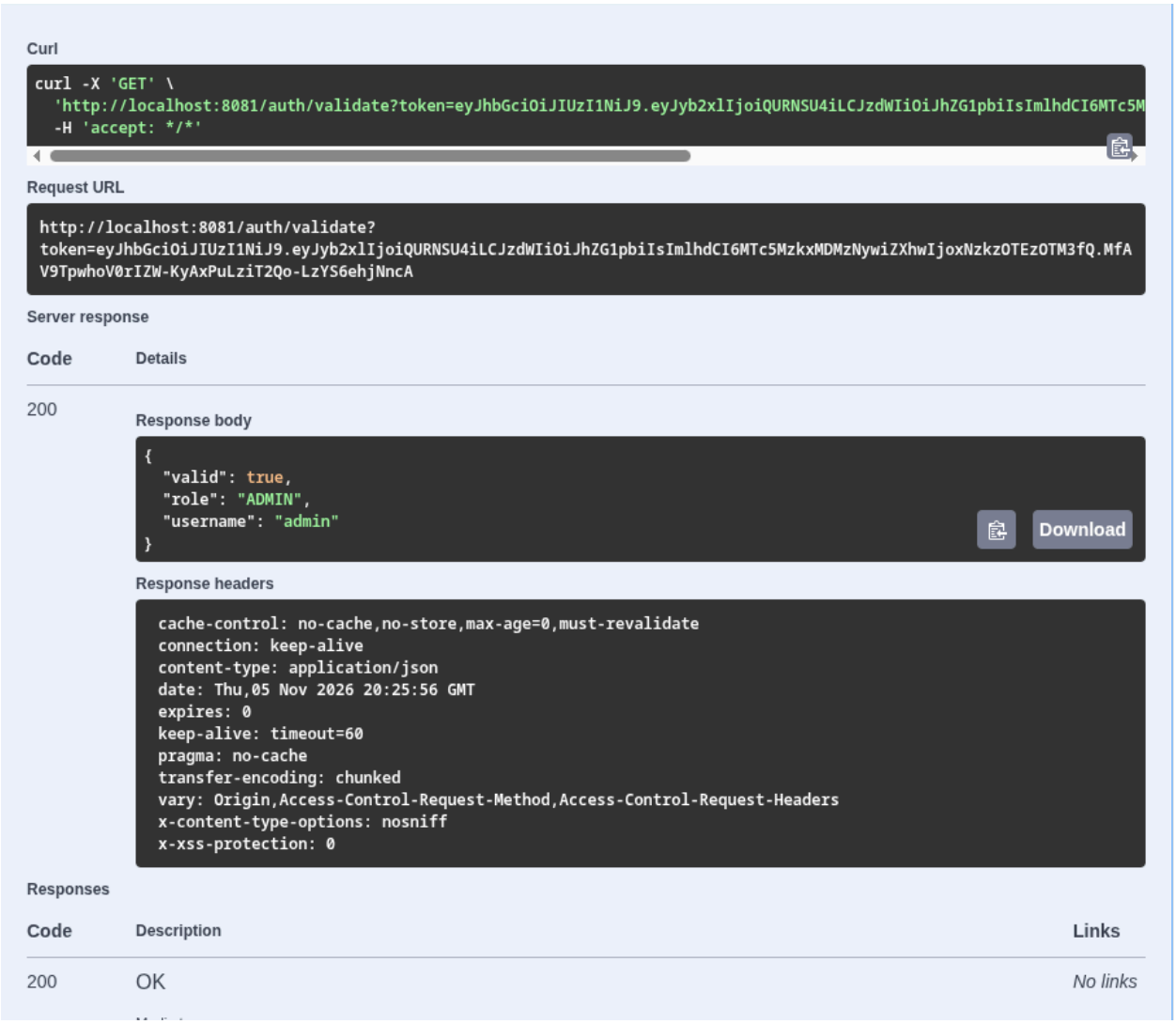


Figura 3 Validación de token JWT

4.4 Consulta de estudiantes con autorización

Se accedió a la interfaz swagger de “consulta-service” y se utilizó la opción **Authorize** para autorizar con un token JWT válido.

Posteriormente, se ejecutó el endpoint “GET /api/estudiantes”. La solicitud fue autorizada y el servicio permitió consultar la información de los estudiantes

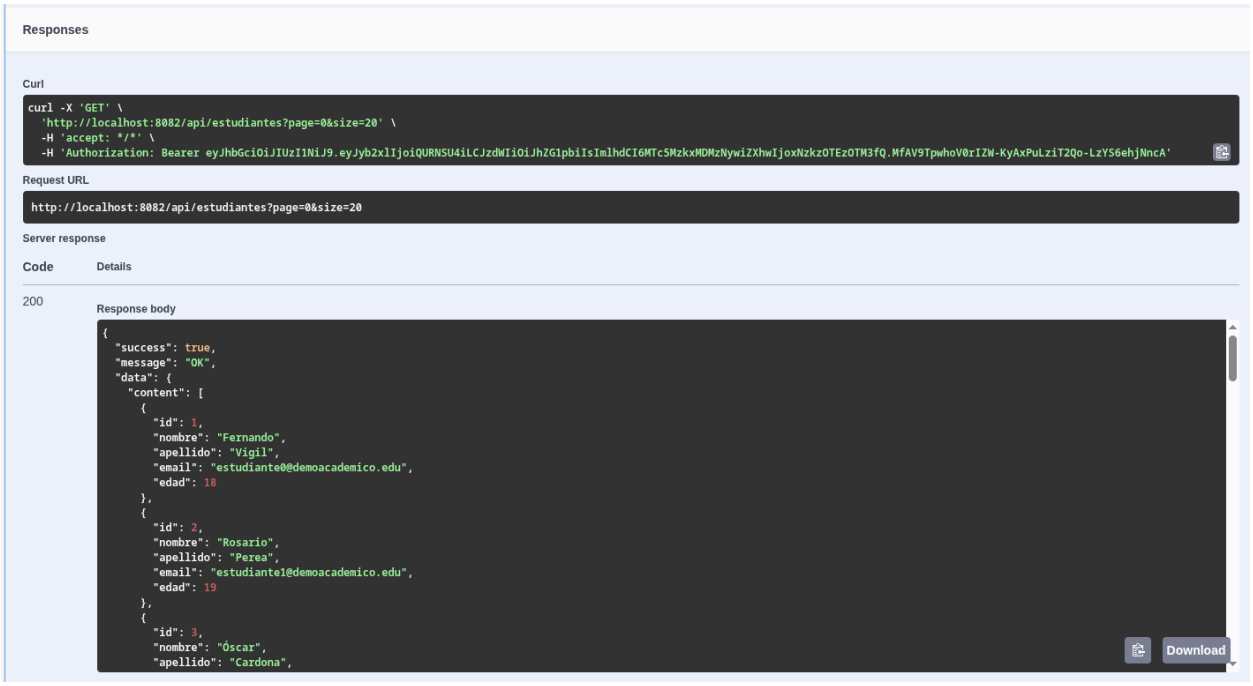


Figura 4 Consulta de estudiantes con token válido

4.5 Consulta de estudiantes sin validación

Se ejecutó el endpoint “GET /api/estudiantes” sin proporcionar un token de autenticación. El servicio rechazó la solicitud debido a que no se presentó una autorización requerida para acceder al recurso.

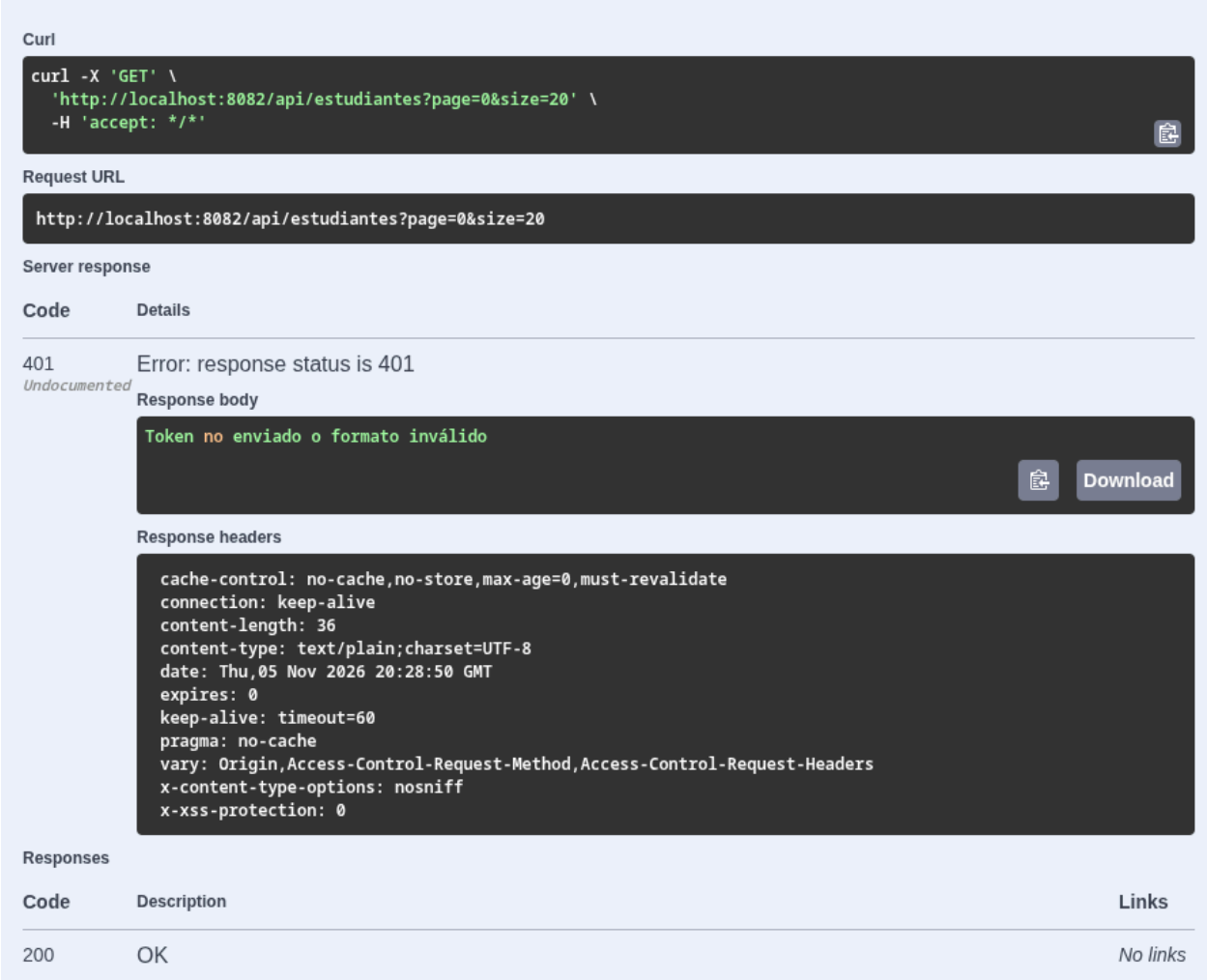


Figura 5 Consulta de estudiantes sin autorización|

4.6 Consulta de estudiantes con token inválido

Se realizó una solicitud al endpoint “GET /api/estudiantes” utilizando un token inválido. El servicio verificó la autenticación y rechazó la solicitud al no poder validar el token proporcionado.

Curl

```
curl -X 'GET' \
  'http://localhost:8082/api/estudiantes?page=0&size=20' \
  -H 'accept: */*' \
  -H 'Authorization: Bearer sfasdasdfsdfsdafsad'
```

Request URL

```
http://localhost:8082/api/estudiantes?page=0&size=20
```

Server response

Code	Details
401 *Undocumented*	Error: response status is 401

Response body

```
Token inválido o expirado
```

Download

Response headers

```
cache-control: no-cache,no-store,max-age=0,must-revalidate
connection: keep-alive
content-length: 26
content-type: text/plain;charset=UTF-8
date: Thu,05 Nov 2026 20:29:29 GMT
expires: 0
keep-alive: timeout=60
pragma: no-cache
vary: Origin,Access-Control-Request-Method,Access-Control-Request-Headers
x-content-type-options: nosniff
x-xss-protection: 0
```

Responses

Code	Description	Links
200	OK	*No links*

Figura 6 Consulta de estudiantes con token inválido

5. Ejecución mediante Docker Compose

Se verificó la construcción y ejecución de los dos microservicios mediante Docker Compose. Cada servicio se ejecutó en su propio contenedor y se expusieron los puertos correspondientes para permitir el acceso desde el entorno local.

La comunicación entre los contenedores se realizó mediante la red interna definida en Docker Compose, utilizando el nombre del servicio de autenticación como dirección de conexión.

```
❯ docker compose ps
WARN[0000] /home/jasu17/Documentos/U/S11/sof3/Lab/Lab01_ing_software_3/microservicios-demoacademico/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion
NAME                IMAGE                                          COMMAND                  SERVICE             CREATED             STATUS              PORTS
auth-service        microservicios-demoacademico-auth-service      "java -jar app.jar"      auth-service        17 seconds ago      Up 16 seconds       0.0.0.0:8081->8081/tcp, [::]:8081->8081/tcp
consulta-service    microservicios-demoacademico-consulta-service  "java -jar app.jar"      consulta-service    17 seconds ago      Up 16 seconds       0.0.0.0:8082->8082/tcp, [::]:8082->8082/tcp
```

Figura 7 Resumen Docker Compose

6. Conclusiones

- La separación de las funcionalidades de autenticación y consulta permitió implementar dos microservicios independientes con responsabilidades específicas.
- La utilización de JWT permitió gestionar la autenticación y controlar el acceso a los endpoints protegidos.
- La comunicación entre los microservicios permitió validar los tokens antes de autorizar las solicitudes de consulta.
- Las pruebas realizadas demostraron el comportamiento esperado ante credenciales válidas e inválidas, así como ante solicitudes autorizadas, no autorizadas y con tokens inválidos.
- Docker Compose facilitó la ejecución conjunta de los servicios y la comunicación entre ellos mediante una red interna.